

Proyecto Decimo Problema

Juan Camilo Solano, Nicolas Hernandez Derchhhhhh, Samuel Montoya Salazar

March 2026

Abstract

En este documento buscamos hacer una demostracion a cabalidad de la existencia de un algoritmo polinomial deterministico para determinar si un numero es primo o no.

1 Historia de como encontrar primos

A lo largo de la historia, los matematicos han desarrollado un interes intrinseco por encontrar numeros primos. Sus propiedades abarcan todas las areas de las matematicas, llegando desde su origen la teoria de numeros hasta a areas en el algebra y geometria. Es por esto que desde los griegos, se busca una manera de encontrar primos de diferentes maneras.

Con el tiempo los matematicos encontraron muchos metodos. El mas antiguo y mas sencillo de aplicar es el metodo de la criba de Eratostenes, que consiste en calcular la raiz cuadrada del numero del que se quiere si es primo o no. Si la raiz es un entero entonces el numero era un cuadrado y por tanto no era primo, de lo contrario se calcula el piso de la raiz y miran los numeros primos menores a este numero, si alguno de estos numeros divide al numero quiere decir que el numero no es primo.

Pero hay algo que todos estos metodos pasan por alto, y es que el numero de numeros primos menores a un numero n es aproximadamente $\frac{\sqrt{n}}{\log(\sqrt{n})}$, lo que hace que el metodo de la criba de Eratostenes tenga una complejidad de $O(\frac{\sqrt{n}}{\log(\sqrt{n})})$ lo que no es de complejidad polinomial.

Lo que quiere decir esto es que, aunque el metodo de un resultado correcto, no es un metodo eficiente para encontrar numeros primos. Otro metodo que se usaba antes era el metodo usando el pequeño teorema de Fermat, que dice que si p es un numero primo y a es un numero entero que no es divisible por p , entonces $a^{p-1} \equiv 1 \pmod{p}$. Este metodo se creia que servia para encontrar primos pues el algoritmo sera elegir un numero a y calcular $a^{n-1} \pmod{n}$, si el resultado es diferente de 1, entonces n no es primo. y se prueba con los a menores a n . Sin embargo, este metodo no es correcto pero si polinomial a diferencia del anterior metodo, pues existen numeros compuestos llamados pseudoprimos de Fermat o los numeros de Carmichael que cumplen con esta propiedad para todos los valores de a con los que sean coprimos con n .

Si bien existen muchos criterios para encontrar numeros primos, como el test de Willson, la idea principal del algoritmo es que el criterio en el cual se basa el algoritmo tiene una cota lo suficientemente buena para lograr una complejidad polinomial menor a la que ya se tenia.

2 Primes is in P

Para iniciar la demostracion de este algoritmo, vamos a demostrar el criterio esencial el cual es muy similar al Pequeño teorema de Fermat, pero con una cota mucho mas estricta.

Teorema.

Sea $a \in \mathbb{Z}, n \in \mathbb{N}, n \geq 2$ y $(a, n) = 1$, entonces n es primo si y solo si $(x + a)^n \equiv x^n + a \pmod{n(x)}$ $\square$

Dem.

Para demostrar la implicacion directa suponga que n es primo entonces por teorema binomial se tiene que:

$$\begin{aligned}(x+a)^n &= \sum_{k=0}^n \binom{n}{k} x^{n-k} a^k \\ &= x^n + a^n \pmod{n(x)}\end{aligned}$$

pues cada termino de $\binom{n}{k}$ con $1 \leq k \leq n-1$ es divisible por n pues siempre esta el coeficiente de n en el numerador. Esto deja que $(x+a)^n \equiv x^n + a^n \pmod{n(x)}$ y por pequeño teorema de Fermat se tiene que $a^n \equiv a \pmod{n}$ y por tanto $(x+a)^n \equiv x^n + a \pmod{n(x)}$.

Para la suficiencia suponga que n es compuesto, entonces existe un primo q tal que $q|n$ y considere $k \in \mathbb{N}$ tal que $q^k || n$. Note ahora que $q^k \nmid \binom{n}{q}$ pues $\binom{n}{q} = \frac{n!}{q!(n-q)!} = \frac{(n-1)!m}{(q-1)!(n-q)!}$ dejando que $q^{k-1}|m \rightarrow q^k \nmid \binom{n}{q}$. Como $(a, n) = 1$ entonces $q^k \nmid a \rightarrow q^k \nmid a^r \forall r \in \mathbb{N}$, dejando que como $(q^k, a^{n-q}) = 1$ entonces el coeficiente en la expansion binomial del termino x^q no va a ser 0 modulo n y por tanto $(x+a)^n - x^n + aa \not\equiv 0 \pmod{n(x)}$ dando a lugar a una contradiccion

□

2.1 Notaciones

Defina el orden multiplicativo de n modulo r como el menor entero positivo k tal que $n^k \equiv 1 \pmod{r}$ equivalente a el orden en $\mathbb{Z}_r^*$ y se denota por $O_r(n)$, es importante aclarar que el orden solo tiene sentido si $(n, r) = 1$.

$\mathbb{Z}_n$ denota el anillo con elementos modulo n , y $\mathbb{F}_p$ denota el campo con p elementos, es decir $\mathbb{Z}_p$ con p primo. Si $h(x)$ es un polinomio de grado d entonces $\mathbb{F}[x]/(h(x))$ denota el cuerpo finito de orden p^d . Decimos que $f(x) \equiv g(x) \pmod{h(x), n}$ para representar que $f(x)$ y $g(x)$ son equivalentes en el anillo $\mathbb{Z}_n[x]/(h(x))$.

$\phi(n)$ denota la funcion de Euler totient, es decir el numero de enteros positivos menores a n que son coprimos con n .

Ahora con este resultado podemos empezar a entrar mas a profundidad en el algoritmo. Si bien es un resultado muy importante en las matematicas su importancia no es proporcional con su dificultad a diferencia de muchas otras cosas dentro del area. Es un poco contraintuitivo que un resultado asi se demore tanto en ser descubierto puesto que el conocimiento de aqui en adelante para demostrar el algoritmo es un conocimiento de pregrado de matematicas y posiblemente menos para ser demostrado en su totalidad. A tal efecto que quienes lo demostraron fueron estudiantes.

2.2 El Algoritmo

Entrada: entero $n > 1$.

1. Si $(n = a^b \text{ Para } a \in \mathbb{N} \text{ y } b > 1)$, retorne **Compuesto**.
2. Encuentre el r mas pequeño tal que $O_r(n) > \log^2 n$.
3. Si $1 < (a, n) < n$ Para algun $a \leq r$, retorne **Compuesto**.
4. Si $n \leq r$, retorne **Primo**.¹
5. Desde $a = 1$ hasta $\lfloor \sqrt{\phi(r)} \log n \rfloor$ haga lo siguiente:
Si $((X+a)^n \not\equiv X^n + a \pmod{X^r - 1, n})$, retorne **Compuesto**;
6. retorne **Primo**.

Ahora vamos a empezar a demostrar los resultados necesarios que nos llevaran al resultado final.

2.3 Demostracion del Algoritmo

Lema.

Sea $L(m) = \text{mcm}(1, 2, \dots, m)$. Para todo $m \geq 7$, se cumple que:

$$L(m) \geq 2^m$$

□

Dem.

La demostración se basa en el estudio de la integral: $I_n = \int_0^1 x^n(1-x)^n dx$ Es un resultado conocido que $L(2n+1)I_n$ es un entero positivo. Dado que $I_n = \frac{(n!)^2}{(2n+1)!}$, tenemos:

$$L(2n+1) \geq \frac{(2n+1)!}{(n!)^2} = (2n+1) \binom{2n}{n} \quad (1)$$

Utilizando la desigualdad para el coeficiente binomial central $\binom{2n}{n} \geq \frac{4^n}{2n}$, para $n \geq 1$, se obtiene:

$$L(2n+1) \geq (2n+1) \frac{4^n}{2n} = \left(1 + \frac{1}{2n}\right) 2^{2n}.$$

Para $m = 2n+1$, esto implica $L(m) > 2^{m-1}$. Un refinamiento de este método mediante el uso de integrales de la forma $\int_0^1 x^{n+k}(1-x)^n dx$ permite fortalecer la cota.

Específicamente, para $m \geq 7$, procedemos por inducción o verificación directa. Para el caso base $m = 7$:

$$L(7) = 420 > 2^7 = 128$$

Para el paso inductivo, se utiliza el hecho de que $L(m+1) = L(m) \cdot \frac{m+1}{\text{mcd}(L(m), m+1)}$. El crecimiento de la función $\psi(m) = \ln L(m)$ está acotado inferiormente por $m \ln 2$ para $m \geq 7$ según el análisis de las integrales de Chebyshev, lo que concluye que:

$$L(m) \geq 2^m$$

□

Ahora vamos a demostrar las dos implicaciones del teorema que van a terminar dando el resultado final. La primera de estas es mucho mas facil que la reciproca y por ello se siguen los siguientes resultados.

Teorema.

Si n es primo entonces el algoritmo retorna **Primo**.

□

Dem.

Como n es primo entonces no se cumple ni el paso 1 ni el 3 por definicion de ser primo. Ahora por el teorema 1 se tiene que n no puede cumplir el paso 5, pues si n es primo entonces se cumple que $(x+a)^n \equiv x^n + a \pmod{n(x)}$ para todo a , y por tanto el algoritmo retorna **Primo** sin importar en que paso lo retorne. □

De ahora en adelante el logaritmo siempre se va a tomar en base 2 a menos que se indique lo contrario

Lema.

$\exists r \leq \max\{3, \lceil \log^5 n \rceil\}$ tal que $O_r(n) > \log^2 n$.

□

Este resultado nos es util pues para el paso 2 del algoritmo necesitamos encontrar este r y es importante que este r no sea tan grande para que el algoritmo siga siendo polinomial.

Dem.

Note que para $n = 2, r = 3$ se cumple la condicion trivialmente pues $\max\{3, \lceil \log^5 n \rceil\} = 3$ y $O_3(2) = 2 > 1 = \log^2(2)$. Suponga que $n > 2$ entonces como $\log n \geq \log 3 \geq \sqrt[5]{10}$ Entonces $\log^5 n \geq 10$ y $\lceil \log^5 n \rceil \geq 10$. Considere $r_1, r_2, \dots, r_t$ tales que $O_{r_i}(n) \leq \log^2 n$ o $r_i | n$, entonces todos los r_i dividen al siguiente producto:

$$n \prod_{i=1}^{\lceil \log^2 n \rceil} (n^{r_i} - 1) \quad (2)$$

puesto que si $r_i|n$ se tiene trivialmente, y si $O_{r_i}(n) \leq \log^2 n$ entonces el orden de r_i debe estar en el intervalo de $[1, \log^2 n]$ entonces, en particular, estan todos los ordenes de todos los r_i que no dividen a n . Por otro lado,

$$P = n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) \leq n^{\log^4 n} \leq 2^{\log^5 n} \quad (3)$$

Dado que $n^i \leq n^{\lfloor \log^2 n \rfloor}$ y

$$n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) \leq n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i) \leq n^{2 \lfloor \log^2 n \rfloor} \leq n^{2 \log^2 n} = n^{\log^4 n} \quad (4)$$

ahora por el lema 1 se tiene que $L(\lfloor \log^5 n \rfloor) \geq 2^{\lfloor \log^5 n \rfloor} > 2^{\log^5 n} = n^{\log^4 n}$, lo que implica que $n \prod_{i=1}^{\lfloor \log^2 n \rfloor} (n^i - 1) < L(\lfloor \log^5 n \rfloor)$, lo que implica que debe existir un $s \leq \lfloor \log^5 n \rfloor$ tal que s no es un r_i , es decir, $O_s(n) > \log^2 n$ y s no divide a n . Ahora si $(s, n) = 1$ queda demostrado el lema, si $(s, n) > 1$ entonces $s|n$ o $O_s(n) \leq \log^2 n$. Defina $r = \frac{s}{(s, n)}$ veamos que $r \notin \{r_1, r_2, \dots, r_t\}$, pues si $r = r_i$ entonces hay dos casos, $r|P$ o $O_{r_i}(n) \leq \log^2 n$. Si $O_{r_i}(n) \leq \log^2 n$ y $(r_i, n) = 1$ entonces defina $d = (s, n)$, entonces $(r, d) = 1$ pues tanto r como d dividen a P entonces como tienen minimo comun divisor 1 entonces $rd|P$ lo que es una contradiccion pues $s = r(s, n)|P$ lo que es una contradiccion pues s no es un r_i . Ahora si $r|n$ entonces veamos que $(r, d) = 1$, suponga que no entonces existe un primo q tal que $q|r$ y $q|d$, pero como $q|s, q|n$ entonces $qd|s$ y $qd|n$ dejando asi que $qd|d$ entonces $q = 1$ contradiccion entonces como $(r, d) = 1$ entonces por el argumento de antes queda que $s|n$ contradiccion.

□

Con este lema podemos ir avanzando en los pasos del algoritmo, sabemos que el paso 1 ya no se cumple pues de lo contrario claramente habria retornado compuesto. Supongamos que el paso 1 no dio como resultado compuesto, ahora por el anterior lema podemos ejecutar el paso 2 sin problemas pues sabemos que existe un r . Ahora como sabemos que $O_r(n) > 1$ entonces por Teorema de Lagrange debe existir un divisor primo de n , p , tal que $O_r(p) > 1$. Ahora $p > r$ pues si $p \leq r$ entonces el paso 3 del algoritmo hubiera retornado compuesto, por otro lado, si $n \leq p$ entonces esto deberia retornar primo pues tenemos un numero cuyo orden no es potencia de ninguno de los anteriores en un $\mathbb{Z}_r^*$ que es un grupo donde n tiene orden $O_r(n)$ talque $O_r(p)|O_r(n)$ pero $O_r(p)$ es el orden del grupo entonces n tiene que ser primo obligatoriamente. Ahora tenemos que $p \geq r$ y por el paso 2 que $(n, r) = 1$, de ahora en adelante tanto p como r van a estar fijos. Sea $l = \lfloor \sqrt{\phi(r)} \log n \rfloor$, el algoritmo ahora en el paso 5 va a verificar l ecuaciones. Suponga que el algoritmo no retorna compuesto en el paso 5 entonces tenemos que

$$(x + a)^n \equiv x^n + a \pmod{(x^r - 1, n)}, \forall a \in [0, l] \quad (5)$$

Pero como $p|n$ entonces tenemos que

$$(x + a)^n \equiv x^n + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (6)$$

Ahora por el teorema 1 se tiene que

$$(x + a)^p \equiv x^p + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (7)$$

Como en un cuerpo de caracterisitica p , elevar a la p es inyectivo, pues

$$f(x) = g(x) \rightarrow f(x) - g(x) = 0 \rightarrow (f(x) - g(x))^p = 0 \rightarrow f(x)^p - g(x)^p = 0 \rightarrow f(x)^p = g(x)^p \pmod{p} \quad (8)$$

Entonces por las ecuaciones anteriores note que

$$(x + a)^{(n/p)p} = (x + a)^n = x^n + a = x^{(n/p)p} + a = (x^{n/p} + a)^p \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (9)$$

Dejando que

$$(x + a)^{(n/p)p} \equiv x^{(n/p)p} + a \pmod{(x^r - 1, p)}, \forall a \in [0, l] \quad (10)$$

Y como sabemos que elevar a la p es injectivo queda el $(x + a)^{(n/p)} \equiv x^{(n/p)} + a \pmod{(x^r - 1, p)}, \forall a \in [0, l]$
Esta propiedad la vamos a llamar introspectividad y va a ser importante mas adelante.

Definición.

Para un polinomio $f(x)$ y un numero $m \in \mathbb{N}$, se dice que $f(x)$ es introspectivo para m si $f(x)^m \equiv f(x^m) \pmod{(x^r - 1, p)}$. $\square$

Lema.

Si m, n son enteros introspectivos para m entonces mn es introspectivo para $f(x)$. $\square$

Dem.

Note que $f(x)^{mn} = (f(x)^m)^n \equiv f(x^m)^n \pmod{(x^r - 1, p)}$, remplazando $x^m = y$ tenemos que $f(y)^n \equiv f(y^n) \pmod{(x^r - 1, p)}$, remplazando $y = x^m$ tenemos que $f(x^m)^n \equiv f((x^m)^n) \pmod{(x^r - 1, p)}$, lo que implica que $f(x)^{mn} \equiv f(x^{mn}) \pmod{(x^r - 1, p)}$ y por tanto mn es introspectivo para $f(x)$. $\square$

3 Implicaciones en la computación

4 Conclusiones

5 Referencias